Operating Systems and System Programming

Fall 2026

Question 1

A user is running a word processor. They open a very large document, and suddenly the entire application becomes unresponsive, the window won't repaint, menus won't open, and clicking does nothing. However, the operating system itself is still fully responsive (other apps work fine, the mouse moves, the clock ticks).

Questions:

a) Is this a process-level or thread-level failure? Justify your answer.

R/ This is a **thread-level failure**, not a process-level one. The word processor process is still alive (it occupies memory, the OS still tracks it), but the thread handling the UI events is blocked, likely waiting on a synchronous I/O operation (loading the large document) or stuck in a long computation. Since the UI thread is blocked, the window cannot repaint or respond to input.

b) What does the OS's continued responsiveness tell you about the relationship between the word processor and the OS?

R/ The OS is a separate set of processes (window manager, scheduler, input handlers) that are independent of the word processor. The word processor blocking does not block the OS because they run in different address spaces and are scheduled independently. The OS remains responsive because its own threads/processes are not blocked.

c) If the application had been designed with multiple threads, how would the behavior differ?

R/ If the application were multithreaded with one thread for UI, another for loading/processing the document the UI thread would remain free to repaint the window and respond to input while the background thread works. The application would appear responsive even during heavy work.

Question 2

A system administrator notices that the process table is filling up with entries that show status Z. These processes consume no CPU or memory but are accumulating. Eventually the system cannot create new processes.

Questions:

a) Is it a case of Process or Thread? Be specific about the case and how it happens.

R/ It is a **zombie process.** It is a process that has terminated but whose parent has not yet called wait() or waitpid() to collect its exit status. The process is dead, it holds no memory or CPU, but its entry remains in the process table.

b) Why does it still occupy an entry in the process table if it's "dead"?

R/ The process table entry is retained because it holds the child's **exit status** and other termination information that the parent must read. Until the parent reads it, the OS cannot free the entry.

c) What is the parent process doing wrong, and how would you fix it?

R/ The parent is **not calling wait()** on its children. The fix is to call wait() or waitpid() after forking, or to install a SIGCHLD handler that reaps children automatically.

d) Could this happen with threads? Why or why not?

R/ No, **threads do not become zombies** in the same sense. When a thread finishes, its resources are reclaimed by the process because it shares the process's address space and exit status is handled at the process level, not per-thread. Threads can be *joinable* or *detached*, but that's a different mechanism, there's no kernel-level zombie thread state equivalent to the process zombie.

Question 3

A user opens Google Chrome with 15 tabs. In Task Manager, they see not one but **15+ separate chrome.exe processes**, each consuming its own memory. Closing one tab kills exactly one of these processes without affecting the others.

Questions:

a) Why would a browser use multiple processes instead of multiple threads for tabs?

R/ Chrome uses **multiple processes** (one per tab, plus a browser process and GPU process) for **isolation and security**. Each tab runs in its own address space so a crash or exploit in one tab cannot affect others.

b) What is the primary advantage of this design?

R/ The primary advantage is **fault isolation and security**. A malicious or buggy page cannot read another tab's memory, and a crash in one tab doesn't bring down the whole browser.

c) What is the primary disadvantage compared to a multithreaded design?

R/ The primary disadvantage is **resource overhead**: each process has its own address space, page tables, file descriptors, and kernel structures, consuming more memory and making context switches more expensive than threads.

d) If one tab crashes, why don't the others crash? Explain in terms of address spaces.

R/ Because each tab runs in its **own address space**. When one process crashes, the OS tears down only that process's memory; the other processes are unaffected because they have separate page tables and cannot access the crashed process's memory.

Question 4

A systems programmer benchmarks two designs for a server handling thousands of concurrent requests:

- **Design A:** One process per request
- **Design B:** One thread per request (within a single process)
- Design B handles significantly more requests per second, but Design A is more resilient, a crash in one request doesn't take down the others.

Questions:

a) Explain why thread creation and context switching are cheaper than process creation and context switching.

R/ Thread creation and context switching are cheaper because threads **share the same address space**. The OS does not need to:

- Create a new page table or switch page tables,
- Duplicate or copy file descriptor tables,
- Set up a new address space.

A process switch requires switching page tables (and often flushing the TLB), which is expensive. A thread switch within the same process only swaps registers, stack pointer, and PC.

b) Why does Design A survive a crash that would kill Design B?

R/ Design A survives a crash because each request runs in its **own address space**. If one process crashes (segfault, memory corruption, unhandled exception), the OS kills only that process; the others continue. In Design B, all threads share one address space, so a crash in one thread (e.g., dereferencing a bad pointer) typically brings down the entire process, all threads die.

d) If you needed both performance *and* isolation, what architectural approach would you choose?

R/ The best approach is a **hybrid**: a **process pool** where each process handles multiple requests using a **thread pool**. This gives isolation between processes (crash in one process doesn't kill others) while keeping thread-level efficiency within each process. This is essentially the model used by modern web servers (e.g., Apache's prefork + worker MPM, or nginx, etc.).

Question 5

Two threads in the same process each increment a shared global variable counter one million times. The programmer expects the final value to be 2,000,000. Instead, it comes out as something less, say, 1,347,219, and the number changes on every run.

Questions:

a) Explain precisely why the result is less than expected.

R/ The increment counter++ is **not atomic**. It compiles into three steps: load counter into a register, increment the register, store it back. If two threads interleave, both load the same value, both increment, both store, one increment is lost. Over a million iterations, thousands of increments are lost.

b) What is the name of this phenomenon?

R/ This is a **race condition**

c) Is this a problem with threads, with the OS scheduler, or with the programmer? Justify.

R/ It is a **programmer problem**, not a thread or OS problem. Threads and the scheduler are behaving correctly, the programmer failed to synchronize access to shared data. The OS provides primitives (mutexes, semaphores, atomics) but does not automatically protect shared variables.

d) What would happen if the same code were run as two separate processes with shared memory instead of two threads? Would the problem persist?

R/ The problem would **persist** if two processes shared memory. Shared memory across processes is subject to the same race conditions. The problem would **not** occur if each process had its own private copy of the counter, but then they would not be incrementing the same variable, so the final value would be 1,000,000 per process, not 2,000,000 total.

Question 6

Two threads belong to the same process. Thread A modifies a global variable, and Thread B immediately sees the change. However, when Thread A declares a local variable inside a function, Thread B cannot see it, even though both threads are in the same process.

Questions:

a) Explain why Thread B can see the global variable but not Thread A's local variable.

R/ Global variables live in the process's **data segment**, which is shared by all threads in the process. Local variables live on each thread's **own stack**, which is private to that thread. Thread B can see the global because it's in shared memory; it cannot see Thread A's local because that local is on Thread A's stack.

b) What parts of memory are shared between threads, and what parts are private?

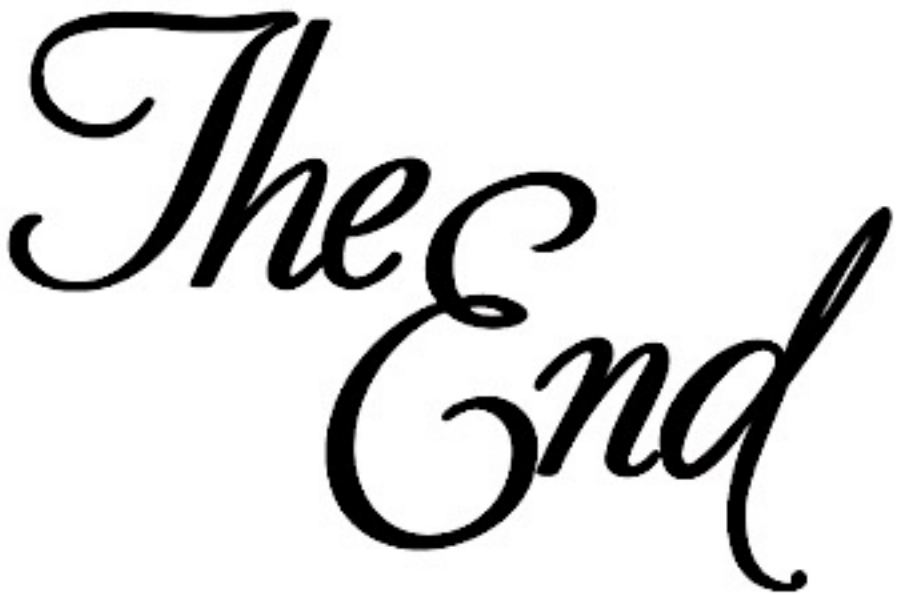
R/ **Shared between threads:** code segment, data segment (globals), heap, open file descriptors, current working directory, signal handlers. **Private to each thread:** stack, registers, program counter, thread-local storage (TLS).

c) If these were two separate processes instead, what would change about sharing the global variable?

R/ If they were separate processes, the global variable would **not** be shared by default. Each process would have its own copy in its own address space. To share, the programmer would need explicit IPC (shared memory, pipes, message queues, etc.).

d) Why is this shared-memory model both a benefit and a danger?

R/ The shared-memory model is a **benefit** because communication is fast, no kernel involvement, no copying, no serialization. It's a **danger** because any thread can corrupt shared data, leading to race conditions, inconsistent state, and bugs that are hard to reproduce. It shifts the burden of synchronization entirely onto the programmer.


The End